# WS5 Build Guide

Oct 23, 2026

## What WS5 owns and what each gate needs

WS5 builds the one screen the customer watches, and owns the story told on it. The fastest way to build it is against **recorded drill evidence**: WS2's `events.jsonl` files and referee results, rather than live systems. The same replay feature then becomes the walkthrough fallback. It beats a video, because it shows real evidence at a pace the presenter controls.

| Gate | What WS5 must show |
|---|---|
| G1 (Oct 26) | The Control Center runs locally on the sample drills; the event catalog has been proposed to WS4; the list of deck corrections is agreed |
| G2 (Nov 2) | Deployed in the lab behind sign-in; shows a real drill from WS2's evidence; the approval and kill buttons are wired to the agreed API |
| G3 (Nov 9) | The full loop runs live from WS4's workflow, including agent tool calls, the verdict and the scorecard |
| Code freeze (Nov 13) | The run of show is rehearsed; a replay fallback is recorded from a real full run; the deck and script are updated |
| Walkthrough (Nov 18) | Two dress rehearsals are done |

### Plan for the days to G1

- **Friday Oct 23:** Phases 0 and 1. Send the proposed event catalog to WS4 and WS2 today, because it's the only thing WS5 needs from them before G2.
- **Weekend (optional) or early Monday:** Phases 2 to 5, run locally, plus the deck corrections (Phase 7, step 1).
- **Monday Oct 26:** the G1 demo replays the sample drill. Once WS2 records its first real drill that morning, replay that one too.

All the code below is also in the starter zip shared with this guide. Steps are numbered within each phase.

## Phase 0: Prerequisites and repo layout

The Control Center is one small Python web app in the same stack as the order service. It has no JavaScript build step, no frontend framework and no CDN. That's faster to build, and leaves less to secure.

1. **Install the tools:** Python 3.12, Docker, Git and a current browser. Node.js isn't needed.
2. **Create the WS5 folders in the shared repo.**

```
src/control-center/
  app/main.py          routes, sign-in guard, live stream, approvals, kill
switch
  app/events.py        event stores (local folder, Blob) with live tail and
replay
  app/stages.py        events to stages, banner, feed and scorecard (pure
functions)
  app/security.py      sign-in modes, CSRF checks, security headers
  app/templates/       base, login, index and drill pages
  app/static/          app.js and app.css
  fixtures/            make_samples.py and two sample drills
  tests/
  requirements.txt
  Dockerfile
infra/modules/control-center.bicep
.github/workflows/control-center.yml
docs/walkthrough/      run-of-show.md, presenter-notes.md, deck-
corrections.md
```

3. **Agree one thing with WS4 now.** In the plan, WS4's workflow later runs inside this same container. For G1 and G2 the UI runs on its own. When WS4 arrives, its workflow should write events through the same store interface in `events.py`, so the UI needs no changes.

## Phase 1: Contracts

The UI reads exactly one thing: a drill's `events.jsonl`, in the line format WS2 already writes:

```
{"ts": "2026-10-26T09:30:05.000Z", "drill_id": "drill-...", "action":
"fault.inject", "result": "ok", "details": {}}
```

It writes only two event types, both requests for someone else to act on: a human decision and a kill request. Send this section to WS4 and WS2 today, saved as `contracts/event-catalog.md`.

1. **Events WS2's scripts already write:**

| `action` | `result` | `details` the UI uses |
|---|---|---|
| `fault.inject`, `fault.restore` | `ok` | `model`, `entries_after` |
| `wait.forbidden`, `wait.ok` | `ok` or `timeout` | `seconds` |
| `referee.<label>`, for example `referee.after-restore` | `PASS` or `FAIL` | the referee result: `last_step`, `error`, `orders_persisted` |
| `recover.broad` | `denied` or `UNEXPECTED_ALLOW` | `code` |
| `reset` | `ok` | none |

2. **Events proposed for WS4, WS3 and WS1 to emit.** The UI already handles all of them. WS4 can rename any of them, as long as this table is updated the same day.

| `action` | Emitted by | `result` | `details` |
|---|---|---|---|
| `drill.start`, `drill.end` | WS4 workflow | `ok` | `mode` |
| `planner.tool_call`, `engineer.tool_call` | WS4, from MCP spans | `allow` or `deny` | `tool`, `endpoint`, `decision`, `summary` |
| `planner.options` | WS4 | `ok` | `options`: a list of `{id, label}` |
| `approval.requested` | WS4 gate | `ok` | `request_id`, `options` |

| action | Emitted by | result | details |
|---|---|---|---|
| approval.decided | **Control Center** | ok | request_id, option_id, option_label, approver, method, latency_s |
| planner.brief | WS4 | ok | summary |
| engineer.change_report | WS4 | ok | branch |
| ci.result | WS4 | pass or fail | run_url |
| deploy.revision | WS2 Ops | ok | image |
| verdict | WS4 | VERIFIED, FALSE_GREEN or ESCALATED_* | none |
| usage | WS4 | ok | tokens, est_cost_usd |
| metrics.orders | WS1 load generator | ok | orders_per_min, failed_total, backlog |
| kill.requested | **Control Center** | ok | by, method |

3. **The approval API.** The workflow posts `approval.requested`, and the Control Center shows the options.
   - A click sends `POST /drills/{drill_id}/approvals` with `{"request_id", "option_id"}`.
   - If the request is still open, the Control Center appends `approval.decided` and returns 202.
   - If no request is open, it returns 409. That prevents double decisions and decisions on stale requests.
   - The Control Center never carries out the action itself. WS4's gate waits for `approval.decided`, then asks the Ops Service, and Azure Policy still has the final say. That's why choosing the broad option produces a visible denial rather than a broad grant.

4. **The kill switch.** `POST /drills/{drill_id}/kill` appends `kill.requested`. WS4's workflow and WS2's Ops Service act on it.
5. **How events map to the nine stages on screen:**

| Stage | Events |
|---|---|
| Baseline | `drill.start`, `referee.baseline`, and `reset` or `wait.ok` before any fault |
| Fault | `fault.inject`, `wait.forbidden`, `referee.during-fault` |
| Investigate | `planner.tool_call` before the False Green, `planner.options` |
| Decide | `approval.*`, `recover.broad` |
| Restore | `fault.restore`, `recover.narrow`, `wait.ok` after the fault |
| False Green | `referee.after-restore` with FAIL |
| Repair | `planner.*` after the False Green, `engineer.*`, `ci.result` |
| Verify | `deploy.revision`, later referee runs, `reset` after the False Green |
| Evidence | `verdict`, `drill.end` |

6. **The banner rules.**
   - A `verdict` event always wins.
   - Otherwise, infrastructure green while the customer journey fails shows **FALSE GREEN**.
   - A failing journey alone shows **CUSTOMERS CAN'T ORDER**.
   - A passing journey after a fault shows **CUSTOMERS ORDERING AGAIN**.

   Only the workflow's verdict can say **VERIFIED**. A recovery by restart in the G1 drill is honest progress, not a verified fix.

# Phase 2: Control Center backend

The whole screen is computed on the server by one pure function, `build_view(events)`, and streamed to the browser after every new event. There's no client-side state to drift. A browser that reconnects recovers by itself, and the same function makes the screen unit-testable.

1. **Copy `src/control-center/` from the starter zip and install it.** `requirements.txt` pins the tested versions:

```
fastapi==0.161.1
uvicorn[standard]==0.54.0
jinja2==3.1.6
python-multipart==0.0.32
itsdangerous==2.2.0
azure-identity==1.26.0
azure-storage-blob==12.31.0
```

2. **Know what `app/events.py` does. It's the only file that knows where events live.**
   - `EVENT_SOURCE=dir:<path>` reads `evidence/<drill-id>/events.jsonl` on disk. That's WS2's format, used for G1 and local work.
   - `EVENT_SOURCE=blob:<account-url>/<container>` reads the same file from Blob storage with the app's managed identity, for G2.
   - `live()` polls every half second and yields the event list whenever it grows. Half-written last lines are skipped until they're complete.
   - `replay()` plays a recorded drill back with its real spacing, sped up, with any single gap capped at 3 seconds. A 90-second propagation wait therefore never stalls the screen.
   - Every drill ID must match `^drill-[A-Za-z0-9-]{1,64}$` before any file or blob is touched, so a URL can't reach outside the evidence folder.

```
async def replay(events: list[dict], speed: float = 10.0, max_gap: float =
3.0):
    """Plays recorded events back with their real spacing, sped up, long
waits capped."""
    shown = []
    previous = None
    for event in events:
        if previous is not None:
            gap = (parse_ts(event["ts"]) -
parse_ts(previous["ts"])).total_seconds() / max(speed, 0.1)
            await asyncio.sleep(min(max(gap, 0.0), max_gap))
        shown.append(event)
        previous = event
        yield list(shown)
```

3. **Know what `app/stages.py` computes.** It applies the stage mapping and banner rules from Phase 1, writes a plain-language line for each event in the activity feed, and calculates the scorecard:

| Scorecard item | Calculated as |
|---|---|
| Verified recovery | From the first `fault.inject` to the first passing referee run after it |
| False Green lasted | From the first `wait.ok` after the fault to that passing run; only shown if `after-restore` failed |
| Recovered by | "candidate fix" if a `deploy.revision` came before the recovery, otherwise "restart" |
| Policy denials | Number of `recover.broad` events with result `denied` |
| Decision time | Total `latency_s` across `approval.decided` events |
| Agent tool calls | Planner and Engineer `tool_call` events, with denials shown in brackets |
| Orders per minute, failed orders, backlog | The latest `metrics.orders` event (from G2) |
| Model tokens, estimated cost | Totals across `usage` events (from G3) |

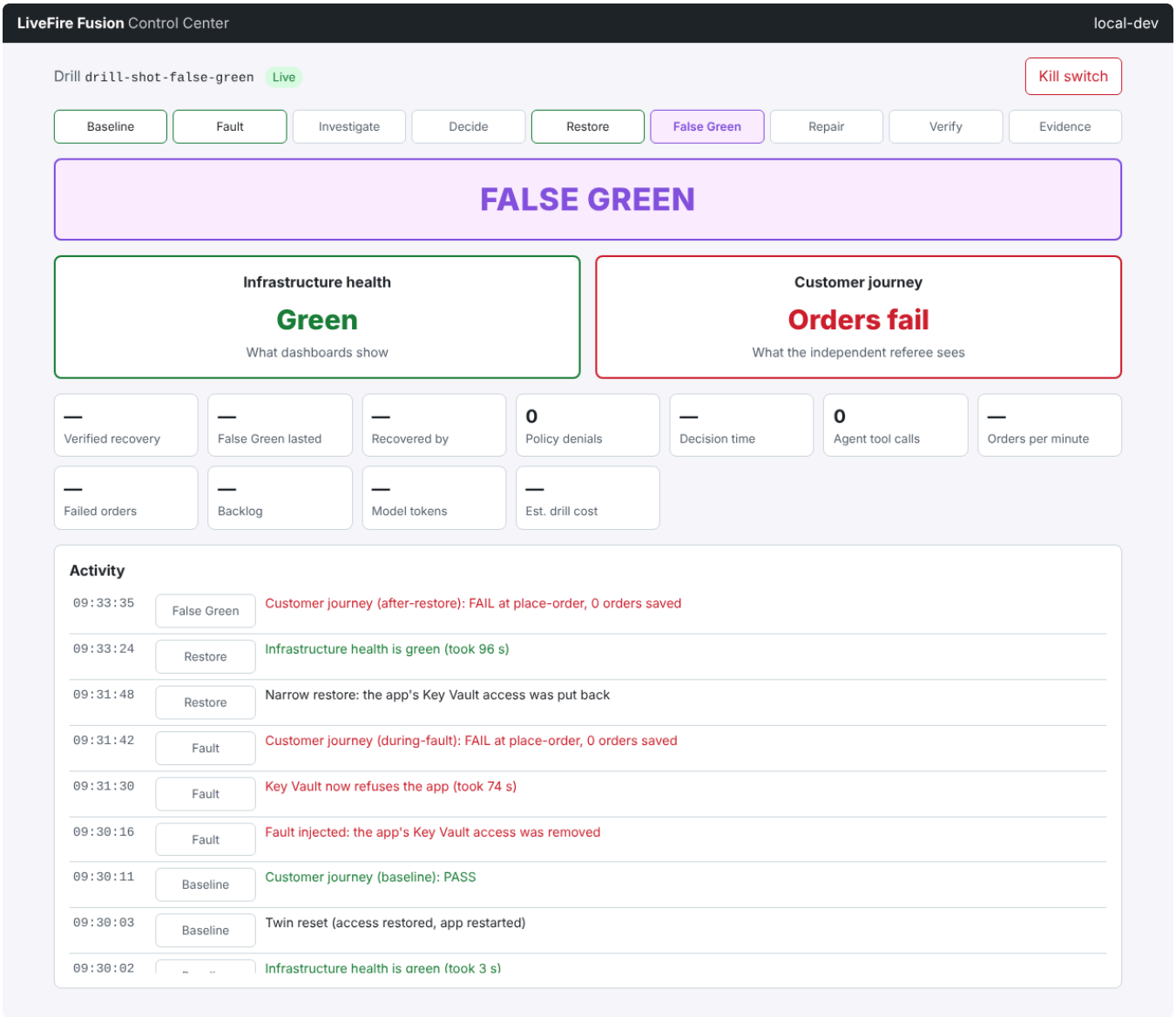
4. **Know the routes in `app/main.py`.**

| Route | Purpose |
|---|---|
| `GET /` | The list of drills in the event store |
| `GET /drills/{id}?mode=live` or `?mode=replay&speed=10` | The drill page |
| `GET /drills/{id}/stream` | Server-sent events: the full view after each new event, then `end` when a replay finishes |
| `GET /api/drills/{id}/view` | The same view as JSON, for tests and for WS4's checks |
| `POST /drills/{id}/approvals` | Records a human decision (CSRF-protected) |
| `POST /drills/{id}/kill` | Records a kill request (CSRF-protected) |
| `GET /login`, `POST /login`, `POST /logout` | Passcode sign-in |
| `GET /health` | Liveness check for Container Apps |

5. **Run the tests:** `pytest -q src/control-center/tests`. All 12 pass on the starter code. They cover:
   - the G1 False Green sequence, the full-loop scorecard and the open-decision state
   - the sign-in guard and the passcode lockout
   - the CSRF and Origin checks
   - drill ID validation
   - a decision recorded once against the signed-in person
   - a check that the browser code never parses HTML, and the live stream's keepalive during quiet periods

## Phase 3: Control Center UI

The drill page answers one question at a glance: can customers order right now? The False Green is the screen's key moment. Infrastructure shows green right next to a failing customer journey, under a banner that names the gap.



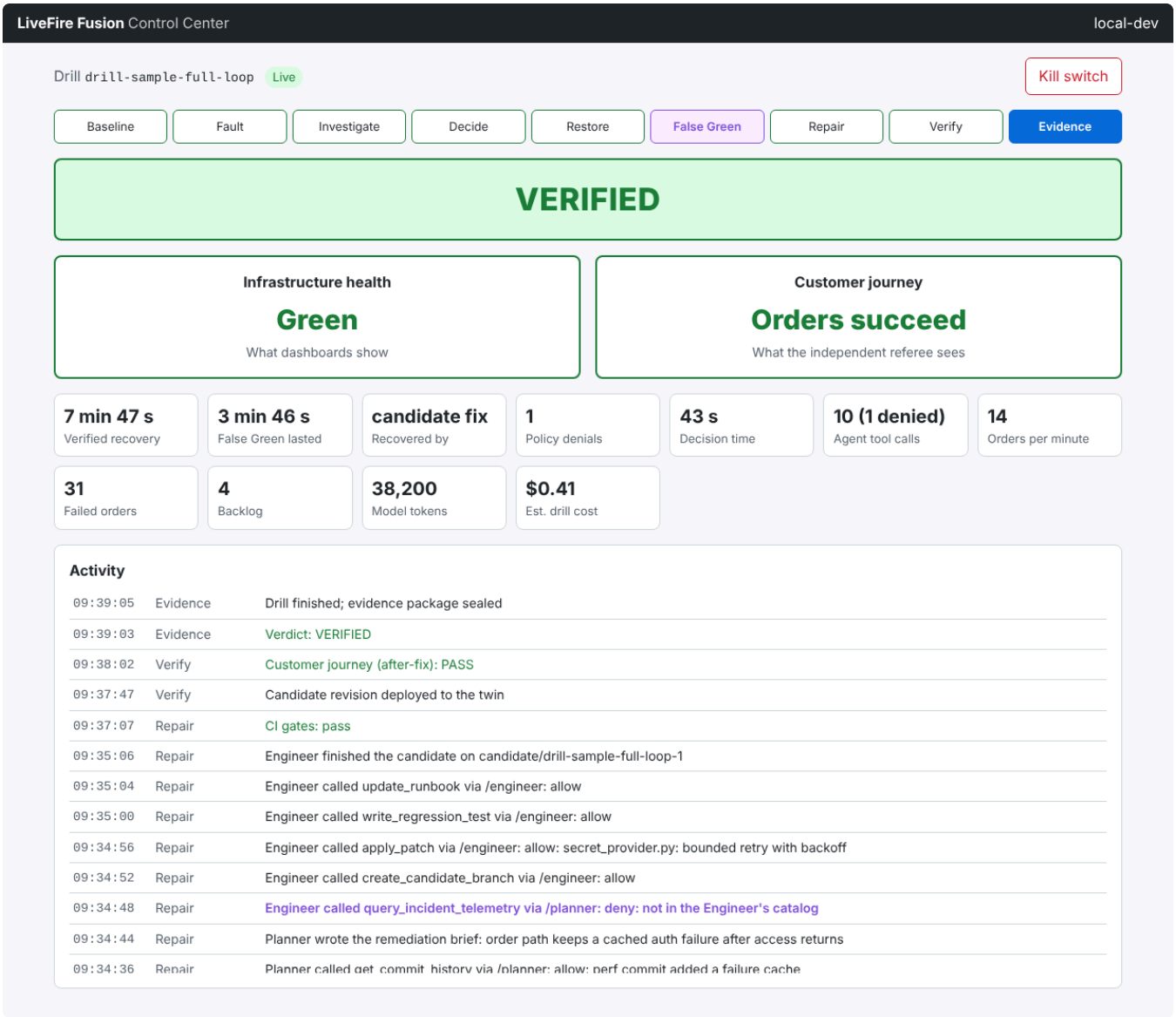

1. **Copy `app/templates/` and `app/static/` from the starter zip.** From top to bottom, the drill page has:
   - **The drill bar:** the drill ID, a Live or Replay badge, and the kill switch.
   - **The nine-stage timeline.** The current stage is blue, finished stages have a green outline, and the False Green stage turns purple.
   - **The banner:** HEALTHY, CUSTOMERS CAN'T ORDER, FALSE GREEN, CUSTOMERS ORDERING AGAIN, or the workflow's verdict.
   - **The split view:** infrastructure health (what dashboards show) next to the customer journey (what the independent referee sees).
   - **Scorecard tiles.** They fill in as the drill produces the numbers, and show a dash until then.

- **The decision card.** It appears only while a decision is open. The broad option is outlined in amber.
- **The activity feed:** newest first, with denials in bold purple, failures in red and passes in green.

2. **Keep the one rendering rule: every value goes in with `textContent`.** Agent summaries and tool output are untrusted text, and they could contain HTML planted through prompt injection. The starter code never uses `innerHTML`, and a test fails the build if anyone adds it.

```
function el(tag, cls, text) {
  const node = document.createElement(tag);
  if (cls) node.className = cls;
  if (text !== undefined && text !== null) node.textContent = String(text);
  return node;
}
```

3. **Load nothing from outside.** That means system fonts, one CSS file and one JavaScript file, with no CDN and no build step. The content security policy in Phase 4 allows only the app's own files, so an external script wouldn't load anyway.
4. **Use replay for rehearsals and as the fallback.** Open `/drills/<drill-id>?mode=replay&speed=10`. Replay disables the decision and kill buttons, and the badge reads Replay, so a recording can't be mistaken for a live run.
5. **Tune it for the room.**
   - Open the page on the actual projector or shared screen, and set the browser zoom so the feed is readable from the back of the room (usually 125 to 150 percent).
   - The banner text always carries the meaning, so the screen doesn't rely on colour alone.
   - Purple was chosen for the False Green so it reads as its own state, separate from red and green.
6. **Check the end state against a full run.** This is the sample full-loop drill at the end, with every scorecard tile filled:




# Phase 4: Security controls

The Control Center is the only part of LiveFire that a customer touches. Its rule is that it can **record** a decision but never **act** on one. Everything consequential still passes through WS4's gate, the Ops Service and Azure Policy. The controls below keep the record trustworthy, and keep the page safe against the untrusted text that agents produce.

1. **Pick the sign-in mode from the day-1 probe.**
   - **`AUTH_MODE=easyauth` (preferred, if app registrations are allowed):**
     - Turn on Container Apps authentication with Microsoft Entra, set to redirect anyone who isn't signed in.

    - In the Entra enterprise application, turn on *Assignment required*, then assign the team and the customer's guest accounts.
    - The app reads the approver's name from `X-MS-CLIENT-PRINCIPAL-NAME`, and trusts that header only in this mode. In any other mode a browser could forge it.
  - **`AUTH_MODE=passcode` (fallback):** a shared passcode plus the person's name, which is recorded on every decision.
    - Generate a long random passcode and store only its SHA-256 hash as a secret.
    - Give the passcode to the customer at the start of the session, and rotate it afterwards.
  - **`AUTH_MODE=local`:** laptop work only. The app refuses to start in this mode unless `ALLOW_LOCAL_AUTH=1` is also set, so it can't be deployed by accident.

```
PASSCODE="$(openssl rand -base64 18)"                    # share this, out
of band, on the day
PASSCODE_SHA256="$(printf '%s' "$PASSCODE" | sha256sum | cut -d' ' -f1)"
SESSION_SECRET="$(openssl rand -base64 48)"
# Store PASSCODE_SHA256 and SESSION_SECRET in the control Key Vault; never
commit them.
```

2. **Copy `app/security.py`.** It holds the sign-in check, the lockout, the CSRF check and the security headers.

```
"""Sign-in, CSRF protection and security headers for the Control Center."""
import hashlib
import hmac
import os
import secrets
import time
from urllib.parse import urlparse

from fastapi import HTTPException, Request

AUTH_MODE = os.environ.get("AUTH_MODE", "passcode")          # easyauth |
passcode | local
if AUTH_MODE not in ("easyauth", "passcode", "local"):
    raise RuntimeError("AUTH_MODE must be easyauth, passcode or local")
if AUTH_MODE == "local" and os.environ.get("ALLOW_LOCAL_AUTH") != "1":
    raise RuntimeError("AUTH_MODE=local needs ALLOW_LOCAL_AUTH=1 and must
never be deployed")
```

```
PUBLIC_PATHS = ("/health", "/login", "/static/")
CSRF_COOKIE = "cc_csrf"
_failures: dict[str, list[float]] = {}

CSP = ("default-src 'self'; script-src 'self'; style-src 'self'; img-src
'self' data:; "
       "connect-src 'self'; frame-ancestors 'none'; base-uri 'none'; form-
action 'self'")
HEADERS = {
    "Content-Security-Policy": CSP,
    "X-Content-Type-Options": "nosniff",
    "Referrer-Policy": "no-referrer",
    "X-Frame-Options": "DENY",
    "Permissions-Policy": "camera=(), microphone=(), geolocation=()",
    "Strict-Transport-Security": "max-age=31536000",
}


def current_user(request: Request) -> dict | None:
    """Who is signed in. Easy Auth headers are trusted only when Easy Auth is
the configured mode."""
    if AUTH_MODE == "easyauth":
        name = request.headers.get("x-ms-client-principal-name")
        return {"name": name, "method": "entra"} if name else None
    if AUTH_MODE == "passcode":
        return request.session.get("user")
    return {"name": "local-dev", "method": "local"}


def check_passcode(passcode: str) -> bool:
    expected = os.environ.get("PASSCODE_SHA256", "")
    if len(expected) != 64:
        return False                    # no passcode configured means
nobody gets in
    actual = hashlib.sha256(passcode.encode()).hexdigest()
    return hmac.compare_digest(actual, expected)


def too_many_failures(ip: str, limit: int = 5, window: float = 300.0) ->
bool:
    now = time.monotonic()
    recent = [t for t in _failures.get(ip, []) if now - t < window]
    _failures[ip] = recent
```

```
    return len(recent) >= limit


def record_failure(ip: str) -> None:
    _failures.setdefault(ip, []).append(time.monotonic())


def csrf_token(request: Request) -> str:
    return request.cookies.get(CSRF_COOKIE) or secrets.token_urlsafe(32)


def require_csrf(request: Request, submitted: str | None) -> None:
    """Double-submit token plus an Origin check on every state-changing
request."""
    cookie = request.cookies.get(CSRF_COOKIE)
    if not cookie or not submitted or not hmac.compare_digest(cookie,
submitted):
        raise HTTPException(status_code=403, detail="CSRF check failed")
    origin = request.headers.get("origin")
    if origin and urlparse(origin).netloc != request.headers.get("host"):
        raise HTTPException(status_code=403, detail="cross-origin request
refused")
```

3. **Know the controls and how each is proven.**

| Control | How it works | Proven by |
|---|---|---|
| Every page and API needs sign-in | A middleware guard redirects to sign-in, or returns 401; only `/health`, `/login` and `/static/` are open | `test_pages_need_sign_in` |
| Passcode guessing is limited | 5 failures in 5 minutes from one address returns 429; the comparison runs in constant time | `test_wrong_passcode_and_lockout` |

| Control | How it works | Proven by |
|---|---|---|
| Sessions can't be read or forged | Signed session cookie: HttpOnly, Secure, SameSite=Strict, 8-hour lifetime | Starlette session middleware |
| No cross-site requests | Every POST needs the CSRF token from an HttpOnly cookie, plus a matching `Origin` | `test_csrf_and_origin_are_enforced` |
| Agent text can't run in the page | `textContent` only; Jinja autoescaping; a strict content security policy with no inline script | `test_browser_code_never_parses_html` and the CSP header test |
| URLs can't reach other files | Drill IDs are validated before any file or blob access | `test_drill_id_cannot_escape_the_evidence_folder` |
| Decisions are attributable and one-off | A decision must match the open request and a listed option; a second decision returns 409; the approver's name and sign-in method are recorded | `test_approval_records_the_signed_in_person_once` |
| No surface beyond the screen | FastAPI's `/docs`, `/redoc` and `/openapi.json` are switched off; the replay speed is clamped | Code review |

| Control | How it works | Proven by |
|---|---|---|
| Clickjacking and data leaks | `frame-ancestors 'none'`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, HSTS | Header test |

4. **Give the app the least privilege it needs (from G2).** Its own identity, `id-lf-control-center`, gets:
   - Storage Blob Data Contributor on the evidence container only, so it can read events and append decisions.
   - Its secrets as Key Vault references.

   It gets **no** rights on the twin, and no route to the Ops Service. The kill switch only records `kill.requested`, and the workflow carries it out. Ask whoever owns the evidence storage to set the immutability policy to allow protected append writes: appends are permitted, overwrites and deletes are not.
5. **Lock down ingress.** HTTPS only (`allowInsecure: false`). Optionally, restrict the app to the presenter's and the customer's network ranges with Container Apps IP restrictions.
6. **Keep secrets out of everything you ship.** Nothing secret goes in the image, the repo or the page. `PASSCODE_SHA256` and `SESSION_SECRET` reach the app as Container Apps secrets read from Key Vault.

## Phase 5: Test locally against sample drills

All of G1 can be done on a laptop. The two sample drills in the starter zip use WS2's exact event format: `drill-sample-g1-manual` is what WS2's scripts produce at G1, and `drill-sample-full-loop` is the G3 target. Their numbers are synthetic.

**Already verified before this guide was published:**

- **Unit tests:** all 12 pass.
- **Browser run (Chromium):**
  - The False Green, open-decision and verified states rendered as in the screenshots, with no console errors.
  - Replay at ×100 passed through FALSE GREEN on its way to VERIFIED.
  - The kill switch was disabled in replay.

- The content security policy blocked an injected script, which confirms it's active.

**Not verified yet:** the Docker build, Easy Auth and the Blob event store. All three need Azure.

1. **Run the tests.** `pytest -q src/control-center/tests`
2. **Regenerate the samples if you change the event catalog.** `python src/control-center/fixtures/make_samples.py`
3. **Start it on the samples.**

```
cd src/control-center/app
AUTH_MODE=local ALLOW_LOCAL_AUTH=1 EVENT_SOURCE=dir:../fixtures/drills \
  uvicorn main:app --port 8080
```

4. **Walk through it.** Open `http://localhost:8080`.
   - The full-loop drill in live mode shows the end state.
   - Replay ×10 tells the whole story in about a minute.
   - The G1 drill ends on CUSTOMERS ORDERING AGAIN, recovered by restart rather than VERIFIED, which is correct before any agents exist.
5. **Try a decision.** Make a copy of the full-loop drill that stops at the first `approval.requested`, then click an option:

```
mkdir -p ../fixtures/drills/drill-try-decision
sed '/"approval.decided"/,$d' ../fixtures/drills/drill-sample-full-
loop/events.jsonl \
  > ../fixtures/drills/drill-try-decision/events.jsonl
```

After a click, the last line of that file should be an `approval.decided` event with your name on it. A second click on the same request is refused with 409.

6. **Try passcode mode.** Set `AUTH_MODE=passcode` with `PASSCODE_SHA256` and `SESSION_SECRET`, as in Phase 4. Browsers accept secure cookies on `http://localhost`, so this works without TLS on a laptop.
7. **Watch a real drill live, which is the G1 demo moment.** Point the app at WS2's evidence folder with `EVENT_SOURCE=dir:<repo>/evidence`. While WS2 runs their Phase 7 drill in another terminal, the screen moves through each stage as the events arrive.

# Phase 6: Image, CI and the Bicep module (for G2)

The build pipeline follows WS1's exactly, so there's nothing new to learn. The deployment is one Bicep module that WS2's `control.bicep` calls once the control Container Apps environment exists.

1. **Write `src/control-center/Dockerfile`.** It needs no system packages, so the image stays small.

```
FROM python:3.12-slim-bookworm
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ .
RUN useradd --create-home --uid 10001 appuser
USER 10001
EXPOSE 8000
# Container Apps ingress is the only way in, so its forwarded headers are
trusted.
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--proxy-
headers", "--forwarded-allow-ips", "*"]
```

2. **Write `.github/workflows/control-center.yml`.** This is WS1's workflow with the paths and image name changed. The tests run on every pull request, and the image is pushed only from main.

```
name: control-center

on:
  push:
    branches: [main]
    paths: ['src/control-center/**', '.github/workflows/control-center.yml']
  pull_request:
    paths: ['src/control-center/**']
  workflow_dispatch:

permissions:
  contents: read
  packages: write

jobs:
  test-build-push:
    runs-on: ubuntu-latest
```

```
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Tests
        run: |
          pip install -r src/control-center/requirements.txt pytest httpx
          pytest -q src/control-center/tests

      - name: Image name (GHCR needs lowercase)
        run: echo "IMAGE=ghcr.io/${GITHUB_REPOSITORY_OWNER,,}/control-center"
>> "$GITHUB_ENV"

      - uses: docker/login-action@v3
        if: github.event_name != 'pull_request'
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - uses: docker/build-push-action@v6
        with:
          context: src/control-center
          push: ${{ github.event_name != 'pull_request' }}
          tags: |
            ${{ env.IMAGE }}:latest
            ${{ env.IMAGE }}:sha-${{ github.sha }}
```

3. **Add `infra/modules/control-center.bicep` from the starter zip.** It compiles cleanly. It deploys:
   - The container app `ca-control-center`, with one replica, HTTPS only, and probes on `/health`.
   - Its secrets as Key Vault references.
   - Two role assignments for `id-lf-control-center`: Key Vault Secrets User on the control vault, and Storage Blob Data Contributor on the evidence container only.
   - Optionally, IP restrictions, through the `allowedIpRanges` parameter.

- When `authMode` is `easyauth`, an Entra sign-in configuration that covers everything except `/health`.

4. **Ask WS2 for four things in `control.bicep`:**
   - The control Container Apps environment. The Ops Service needs it too.
   - The `id-lf-control-center` identity.
   - The control Key Vault.
   - The evidence storage account with its `evidence` container.

   Then call the module:

```
module controlCenter 'control-center.bicep' = {
  name: 'control-center'
  params: {
    location: location
    environmentId: controlEnv.id
    image: controlCenterImage                // ghcr.io/<org>/control-
center:sha-<commit>
    identityId: ccIdentity.id
    identityClientId: ccIdentity.properties.clientId
    identityPrincipalId: ccIdentity.properties.principalId
    authMode: 'passcode'                     // 'easyauth' if the probe
allowed app registrations
    keyVaultName: controlVault.name
    evidenceAccountName: evidence.name
    allowedIpRanges: []                      // e.g. ['203.0.113.0/24'] on
the day
  }
}
```

5. **Put the secrets in the control Key Vault before deploying.** Store `cc-session-secret`, and either `cc-passcode-sha256` or `cc-entra-client-secret` depending on the mode, using the values from Phase 4.
6. **For Easy Auth, register the app once.**
   - Create an Entra app registration with the redirect URI `https://<control-center-fqdn>/.auth/login/aad/callback`.
   - Store its client secret as `cc-entra-client-secret`, and pass its client ID as `entraClientId`.
   - Turn on *Assignment required* for the enterprise application, and assign the team and the customer's guests.

7. **Smoke-test the deployed app.**
   - Signing in works.
   - A drill WS2 has written to Blob storage appears in the list.
   - The browser's developer tools show the `Content-Security-Policy` header and no console errors.

## Phase 7: The story

WS5 also owns what's said while the screen runs. Write each item below as a file in `docs/walkthrough/`, so it's versioned with the code and rehearsed against the real screen.

1. **Deck corrections, `deck-corrections.md` (agree by G1).** Take the list from the implementation plan's Deck section:
   - GPT-4o is retired, so name the deployed GPT-5.x model.
   - Rename Azure AI Agent Service to Microsoft Foundry Agent Service, and add Agent Framework and Toolboxes.
   - Trim the technology stack.
   - Use one line on autonomy for both the deck and the blueprint.
   - Fix "the Planner restores access" to "the workflow restores access after approval".
   - Regenerate both architecture images, which have garbled labels. The plan's architecture diagram can replace them.
   - Fix the "Livefile" typo.
   - Use one timing story throughout.
2. **Run of show, `run-of-show.md`.** Start from the plan's 45-minute table, and add two columns: which tab is on screen, and who's at the keyboard.

| Minutes | On screen | Keyboard |
|---|---|---|
| 0 to 5 | Control Center: HEALTHY, Baseline stage | Presenter |
| 5 to 10 | Control Center: Fault stage, CUSTOMERS CAN'T ORDER | Driver starts the drill |
| 10 to 17 | Control Center feed, then the Foundry trace of the Planner's tool calls | Driver switches tabs |
| 17 to 22 | Control Center decision card | **The customer** clicks |

| Minutes | On screen | Keyboard |
|---|---|---|
| 22 to 27 | Control Center: FALSE GREEN split view | Presenter |
| 27 to 35 | Feed of Engineer calls, then the GitHub branch and CI run | Driver switches tabs |
| 35 to 40 | Control Center: Verify stage, VERIFIED | Presenter |
| 40 to 45 | Scorecard tiles, evidence package, unmerged pull request | Presenter |

3. **Presenter notes, `presenter-notes.md`.** Write three sentences per segment at most, built around the plan's key lines:
   - "We measure recovery in customer transactions."
   - "Policy, not the model, decides what runs."
   - "This is the False Green."
   - "Evidence, not the agent, declares recovery."

   Also include the two honesty lines the blueprint asks for:
   - This is a controlled identity disruption that rehearses a cyber-recovery control, not a simulated attack.
   - The verdict covers this scenario and this journey only.
4. **Roles on the day.** Fix these at the first rehearsal, not on the day:
   - **Presenter:** talks, and owns the timing.
   - **Driver:** runs the drill and switches tabs.
   - **Backup:** watches the lab clock and keeps the replay ready.
   - **Customer:** makes the decision.
5. **Agree the fallback ladder.** Each step is tried before the next, and a replay is always announced as one.
   a. Live drill as planned.
   b. A stage stalls for more than 2 minutes: the presenter covers with the notes while the driver keeps going.
   c. The lab dies or a stage fails: switch the same screen to `?mode=replay&speed=5` of the last full rehearsal, and say "this is the run we recorded on <date>".
   d. Last resort: the screen recording of a full live run.

6. **Record the fallbacks at code freeze (Nov 13).** Copy the evidence folder of the best full rehearsal to `fixtures/drills/drill-rehearsal-<date>`, so replay can show it without a lab. Also capture a screen recording of one complete live run.
7. **Rehearse twice before Nov 18.** Run with a timer, write down the length of each segment, and fix the longest segment before the second rehearsal.
8. **Prepare five answers for Q&A.** Each should take under 30 seconds:
   - **"Does this touch production?"** No. Only a disposable twin, and no production identity exists.
   - **"What if the agent is wrong?"** An independent referee decides, budgets cap the attempts, and a human reviews the pull request.
   - **"What stops it giving itself admin rights?"** Azure Policy, a limited RBAC role and separate MCP scopes. You just watched the broad option get denied.
   - **"Would it work on our application?"** It needs a readiness check: telemetry, a repeatable customer journey, isolation and repository access.
   - **"What does a drill cost?"** Point to the scorecard's token and cost tiles.
9. **Film the recorded submission from replay.** Keep the plan's 3-minute script, and film it from a replay at ×10 to ×20 so the pacing is under your control.

## Phase 8: Wiring in live data for G2 and G3

Until G3, everything reaches the screen through the event store, so wiring a new source means "write events in the catalog's format" and nothing else. The most efficient move is to have every component write events through one shared module.

**For G2 (Nov 2)**

1. **Share the event code.** Move `events.py` to `src/shared/livefire_events.py`. The Control Center, WS2's Ops Service, WS1's load generator and WS4's workflow then all write events through `store.append()`. That gives one format, one validator, and one place that knows about Blob storage.
2. **Send the evidence to Blob.**
   - The Ops Service writes each drill's events with the shared Blob store, and the deployed Control Center reads them with `EVENT_SOURCE=blob:...`.
   - Until the Ops Service exists, run the Control Center on the driver's laptop against WS2's local evidence folder. That works today.

3. **Get business numbers from the load generator.** WS1's load generator emits `metrics.orders` every 30 seconds into the current drill, which fills the orders per minute, failed orders and backlog tiles. The UI needs no change.
4. **Make the decision live before WS4 exists, with `scripts/decide-and-recover.sh`** (in the starter zip, tested here against stubbed scripts).
   - It posts `approval.requested` with the broad and narrow options and waits for the click.
   - If the customer picks broad, it runs WS2's `recover.sh broad`, which is denied and logged, then asks again with only the narrow option.
   - It finishes with `recover.sh narrow`.

   Run it as soon as WS2's drill reaches the Restore step.

```
export DRILL_ID=<the drill in progress>
./scripts/decide-and-recover.sh
```

5. **Make the kill switch act.** Until WS4 exists, a watcher on the driver's machine runs WS2's `reset.sh` when a `kill.requested` event appears. It uses the same polling pattern as the bridge script. From G3, WS4's workflow owns that reaction.
6. **Deploy to the lab** with the Phase 6 module, and run the Phase 6 smoke test.

**For G3 (Nov 9)**

7. **Let WS4's workflow emit the rest of the catalog.** That's the planner and engineer tool calls, options, brief, change report, CI result, verdict and usage, each written through the shared store. Check each new event type against the sample `drill-sample-full-loop` replay. If the live screen and the sample match, the wiring is right.
8. **Retire the bridge.** WS4's gate posts `approval.requested` and waits for `approval.decided` itself. Agree a timeout with WS4 for the case where nobody decides, after which the drill escalates rather than waiting forever.
9. **Add trace and evidence links.** Add `trace_url` to tool-call events, and a final `package.ready` event carrying the evidence package and pull request links. Render each link only if it starts with an allowed prefix, such as the Foundry portal or your GitHub org. It must still go in through a DOM attribute, never as HTML.
10. **Run the full rehearsal on the deployed app.** The scorecard tiles should all be filled from real events, matching the sample's end state.

# Hand-offs and gate checklists

WS5 depends on other workstreams only through the event catalog and one Bicep request, so send both early and keep everything else local until G2.

### Hand-offs

| When | To | What |
|---|---|---|
| Today | WS4, WS2 | `contracts/event-catalog.md` (Phase 1), including the approval and kill APIs |
| Today | Lead | `deck-corrections.md` and the proposed day-of roles |
| By Oct 28 | WS2 | Phase 6's `control.bicep` request: control environment, `id-lf-control-center`, control Key Vault, and an evidence container that allows protected appends |
| By Oct 28 | WS1 | `metrics.orders` events from the load generator |
| Monday Oct 26 | From WS2 | The evidence folder of the first real drill, so it can be replayed |

### G1 (Oct 26)

- [ ] Control Center runs locally on both sample drills; all 12 tests pass
- [ ] Event catalog committed and sent to WS4 and WS2
- [ ] Deck corrections agreed with the lead
- [ ] WS2's first real drill shown live or replayed, if it exists by Monday

### G2 (Nov 2)

- [ ] Shared events module in use by the Ops Service
- [ ] Deployed in the lab behind sign-in, smoke test passed
- [ ] A real drill shown from Blob storage
- [ ] Decision and kill switch wired, through the bridge or WS4
- [ ] Business tiles filled by the load generator

### G3 (Nov 9)

- [ ] Full loop shown live from WS4's workflow, with every tile filled from real events

- [ ] Bridge script retired

**Code freeze (Nov 13)**

- [ ] `run-of-show.md` and `presenter-notes.md` final
- [ ] Best rehearsal saved as a replay fixture; screen recording saved
- [ ] Deck and script updated

**Walkthrough (Nov 18)**

- [ ] Two dress rehearsals done
- [ ] Passcode rotated, or customer guests assigned in Entra; IP ranges set if used

## Gotchas

Most risks to the screen come from other teams' data drifting from the contract, or from a security shortcut taken under time pressure. The first three cause most surprises.

| Gotcha | What happens | Prevention |
|---|---|---|
| Event names drift from the catalog | The feed shows raw action names and the banner or stages go wrong | The catalog is the contract: update it the same day, regenerate the samples, and compare a replay |
| A replay mistaken for a live run | Credibility is lost the moment someone notices | The Replay badge and disabled buttons are built in; also say it out loud |
| VERIFIED shown for a restart | Overclaims what happened | Built in: only the workflow's `verdict` can show VERIFIED |
| More than one replica | Sign-in lockout counts and live streams split across copies | One replica (set in the module) |
| A proxy buffers or closes the stream | The screen updates in bursts or freezes | `X-Accel-Buffering: no` and a 15-second keepalive are built in; test on the customer's network before the day |

| Gotcha | What happens | Prevention |
|---|---|---|
| Trusting the Easy Auth header in passcode mode | Anyone could forge an approver name | The code trusts that header only in `easyauth` mode; don't change that |
| Someone adds `innerHTML` or a CDN script | An opening for script injection; the CSP blocks the script anyway | The test fails the build; keep every asset local |
| Key Vault references before role propagation | The first revision fails to start with secret errors | Wait a few minutes and redeploy, or deploy the roles in an earlier step |
| Immutable evidence without protected appends | Decisions can't be recorded | Turn on protected append writes for the evidence container's immutability policy |
| The lab deallocates | The deployed Control Center disappears | Keep the laptop version and a local copy of the evidence ready as the backup |
| Secure cookies on plain HTTP away from localhost | Sign-in loops | Use the HTTPS app URL, or `localhost` |
| The same passcode across sessions | An old guest can still get in | Rotate the passcode after every walkthrough |